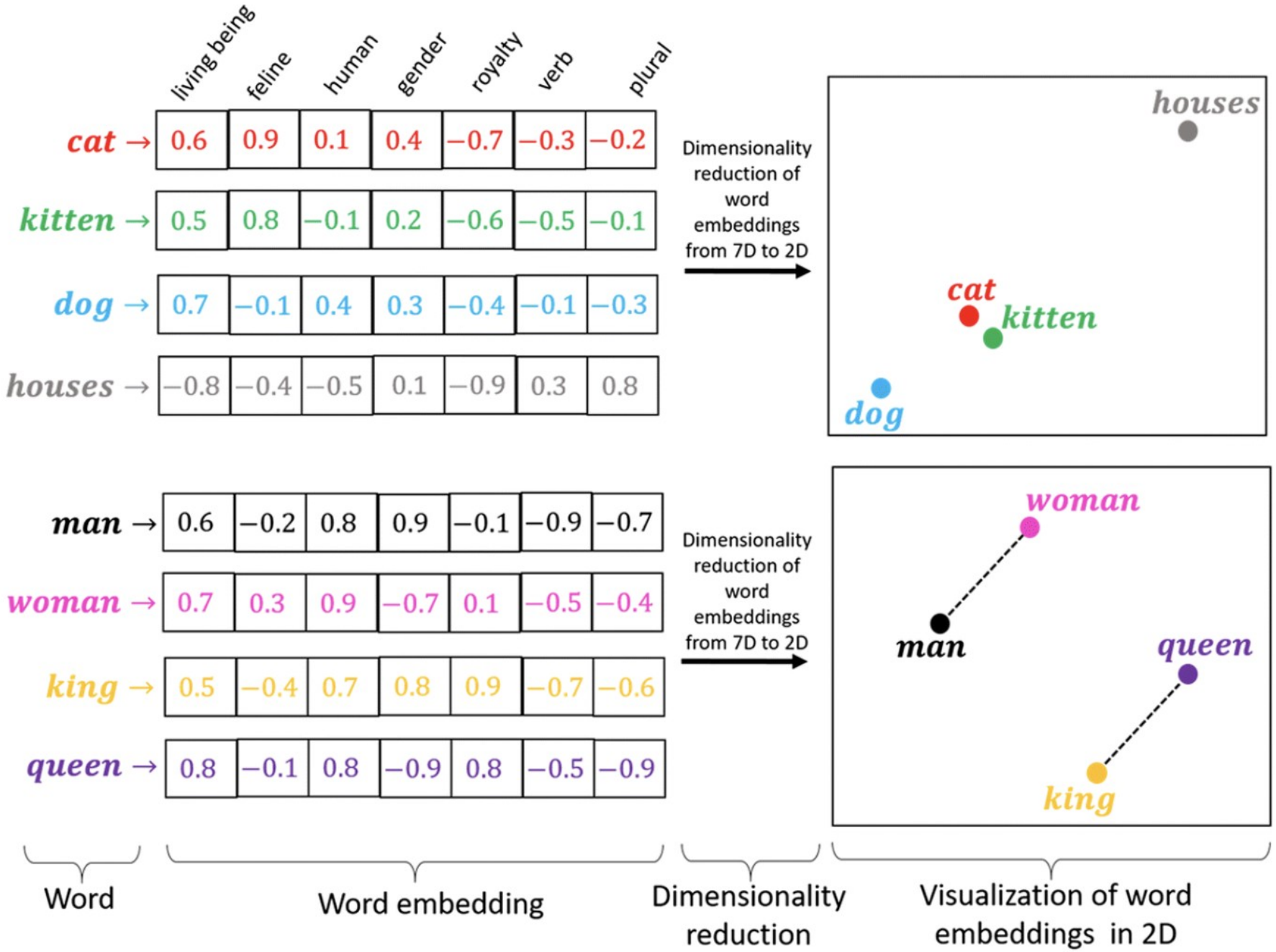
Word embedding

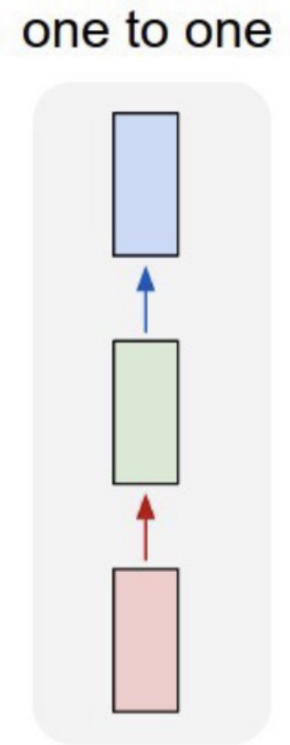
A word like bank, called a **token** when it represents a fundamental piece of text, is commonly encoded as a vector of real, continuous values: the **embedding vector**.

Determining the values inside the embedding vector of a token is a large part of the heavy lifting in text processing. Thankfully, with hundreds of dimensions available to organize the vocabulary of known tokens, embeddings can be pretrained to relate numerically in ways that reflect how their tokens relate in natural language.


living being
feline
human
gender
royalty
verb
plural
cat → 0.6 0.9 0.1 0.4 −0.7 −0.3 −0.2
kitten → 0.5 0.8 −0.1 0.2 −0.6 −0.5 −0.1
dog → 0.7 −0.1 0.4 0.3 −0.4 −0.1 −0.3
houses → −0.8 −0.4 −0.5 0.1 −0.9 0.3 0.8
Dimensionality reduction of word embeddings from 7D to 2D
houses
cat
kitten
dog
man → 0.6 −0.2 0.8 0.9 −0.1 −0.9 −0.7
woman → 0.7 0.3 0.9 −0.7 0.1 −0.5 −0.4
king → 0.5 −0.4 0.7 0.8 0.9 −0.7 −0.6
queen → 0.8 −0.1 0.8 −0.9 0.8 −0.5 −0.9
Dimensionality reduction of word embeddings from 7D to 2D
woman
man
queen
king
Word
Word embedding
Dimensionality reduction
Visualization of word embeddings in 2D

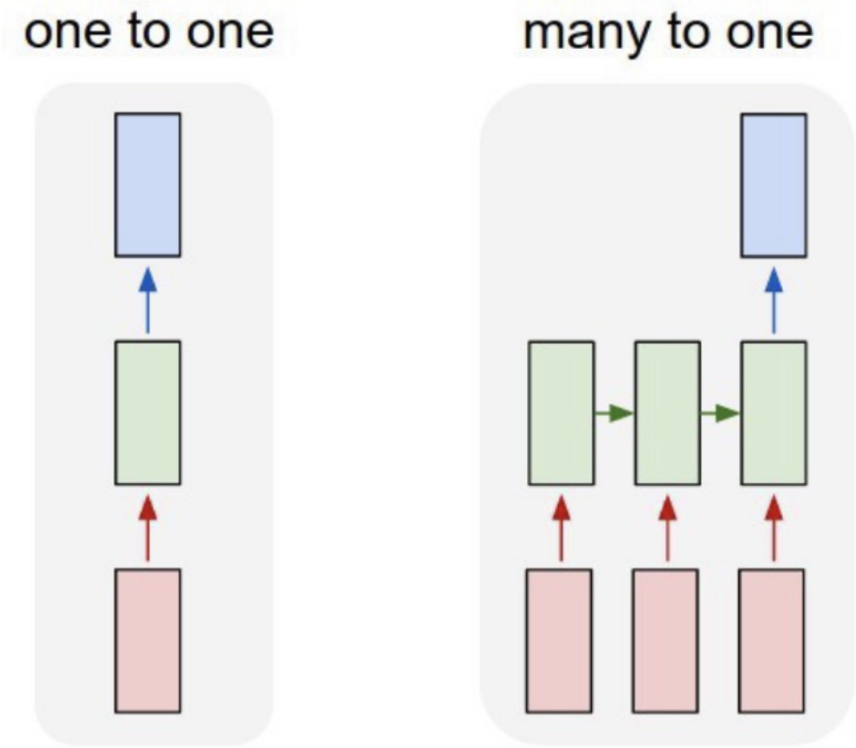
Sequence to sequence models

How to model sequential data?



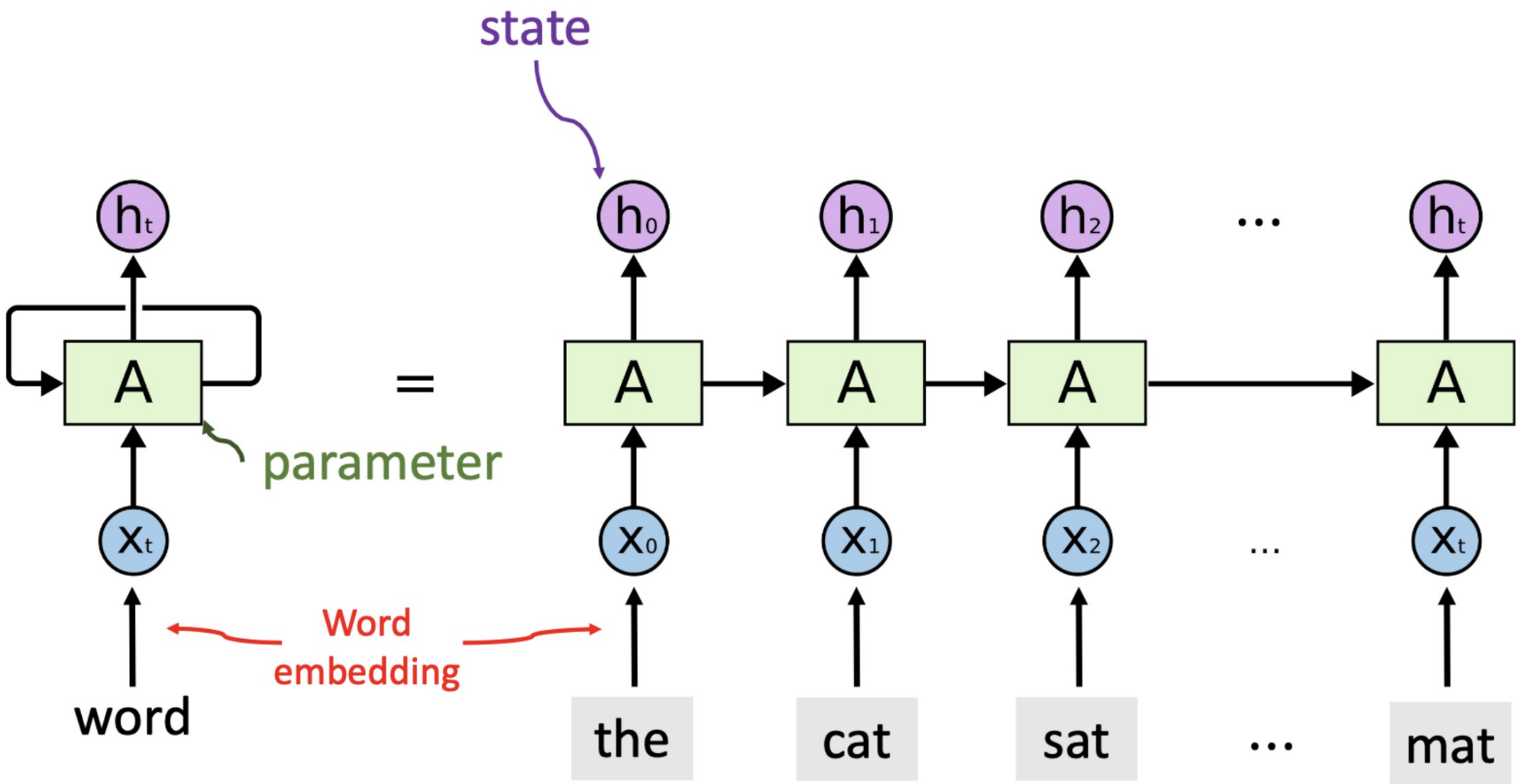
- Limitations of FC Nets and ConvNets:
  - Process a paragraph as a whole.
  - Fixed-size input (e.g., image).
  - Fixed-size output (e.g., predicted probabilities).

How to model sequential data?

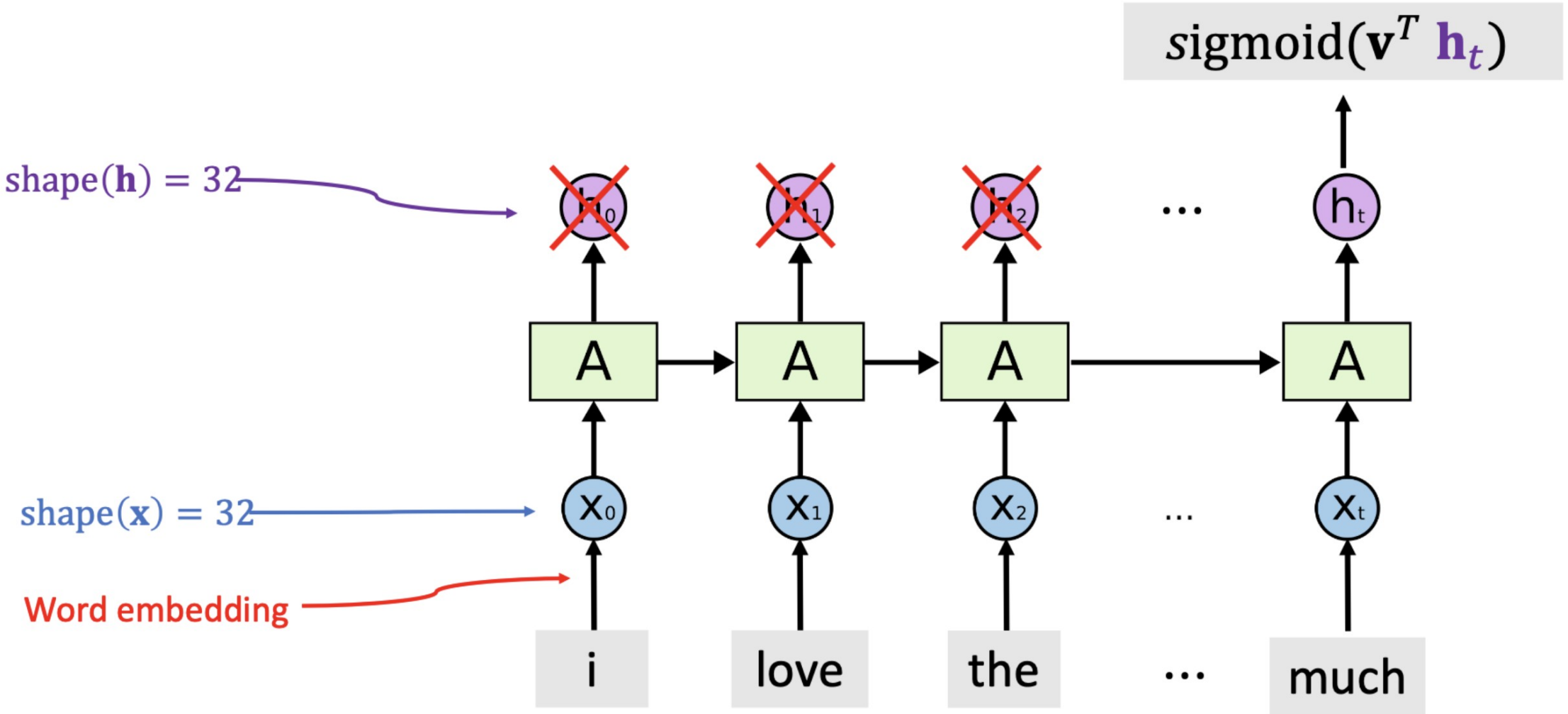


- Limitations of FC Nets and ConvNets:
  - Process a paragraph as a whole.
  - Fixed-size input (e.g., image).
  - Fixed-size output (e.g., predicted probabilities).
- RNNs are better ways to model the sequential data (e.g., text, speech, and time series).

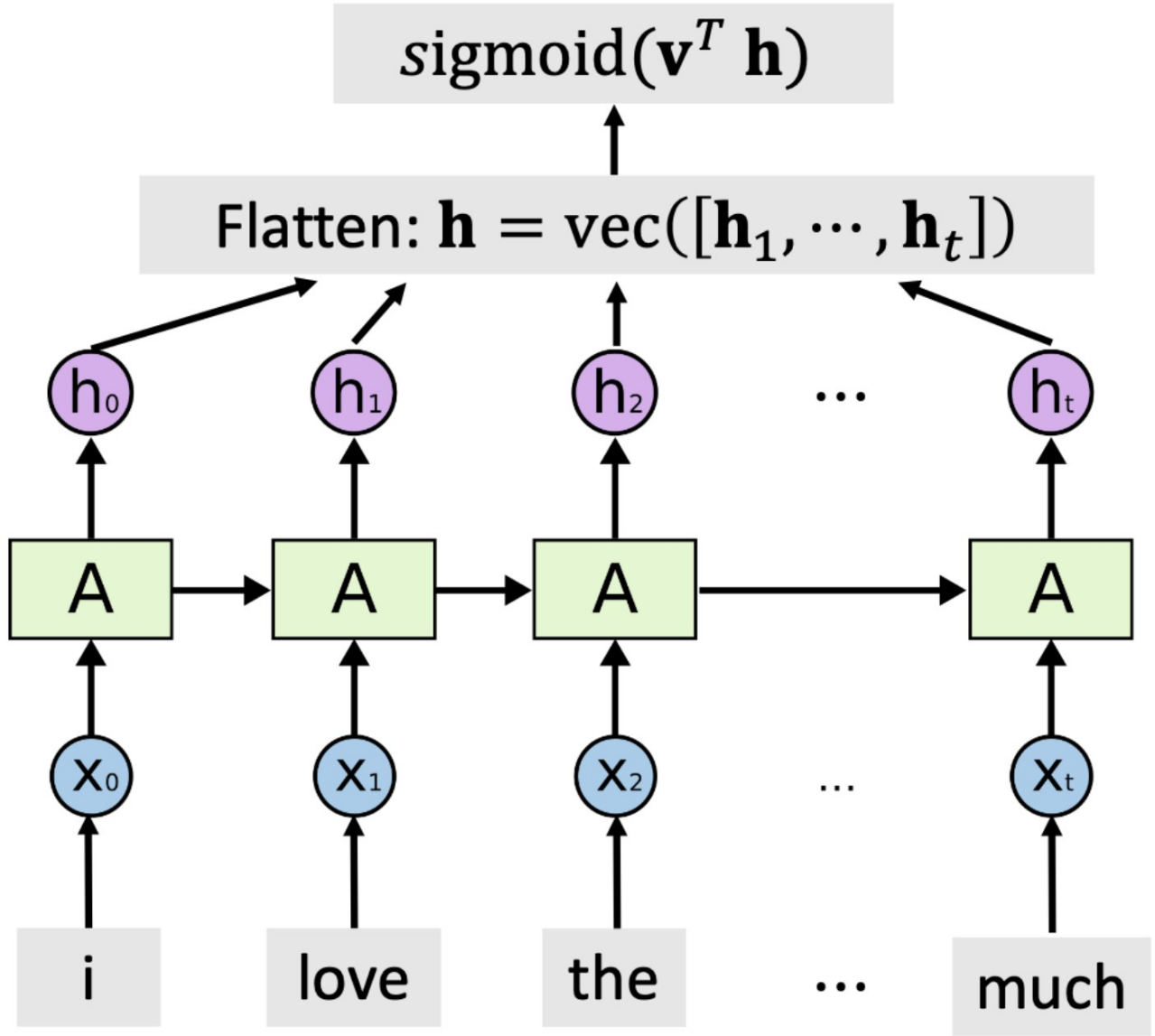
Recurrent Neural Networks (RNNs)



Simple RNN for IMDB Review

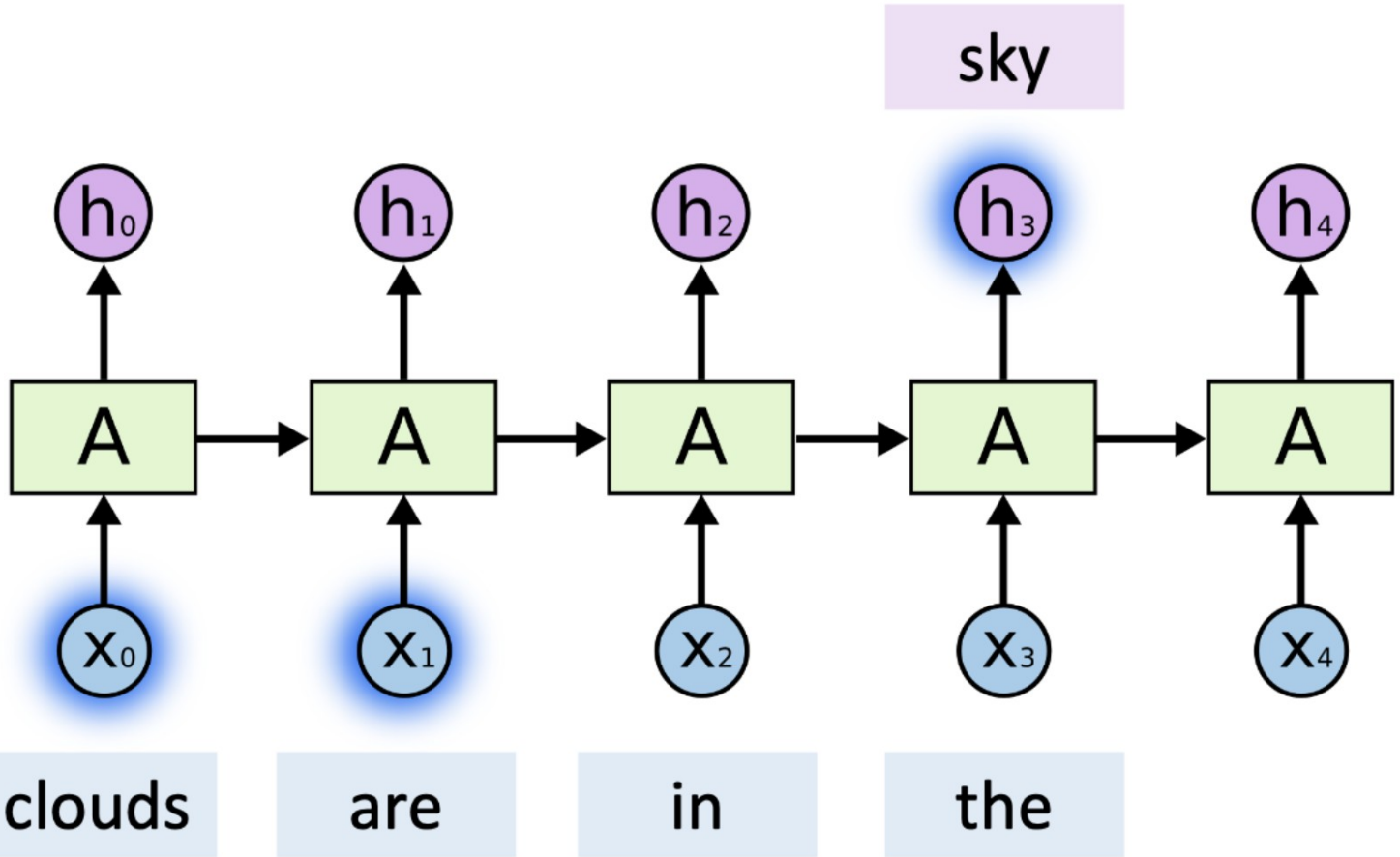


Simple RNN for IMDB Review



SimpleRNN is good at short-term dependence.

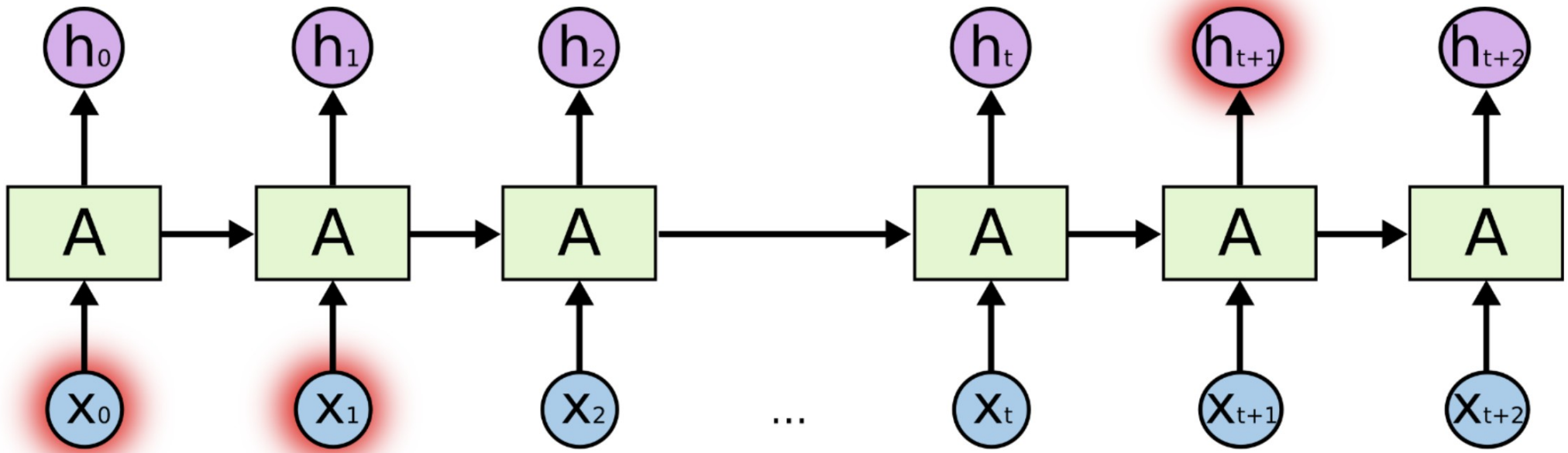
Predicted next words:



Input text:

Figures are from Christopher Olah's blog: Understanding LSTM Networks.

SimpleRNN is bad at long-term dependence.

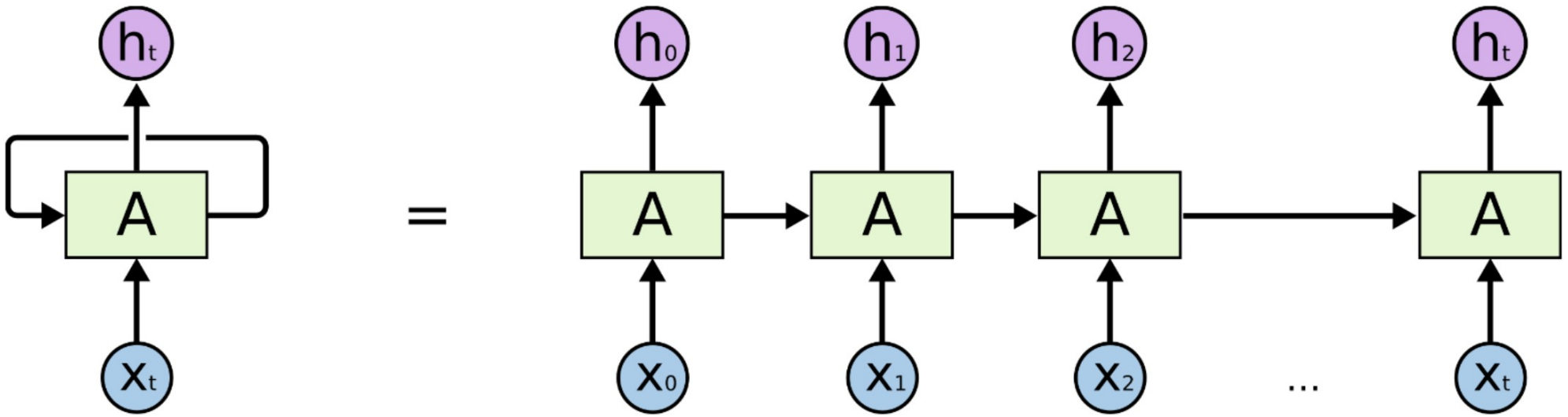


$\mathbf{h}_{100}$ is almost irrelevant to $\mathbf{x}_1$: $\frac{\partial \mathbf{h}_{100}}{\partial \mathbf{x}_1}$ is near zero.

Figures are from Christopher Olah's blog: Understanding LSTM Networks.

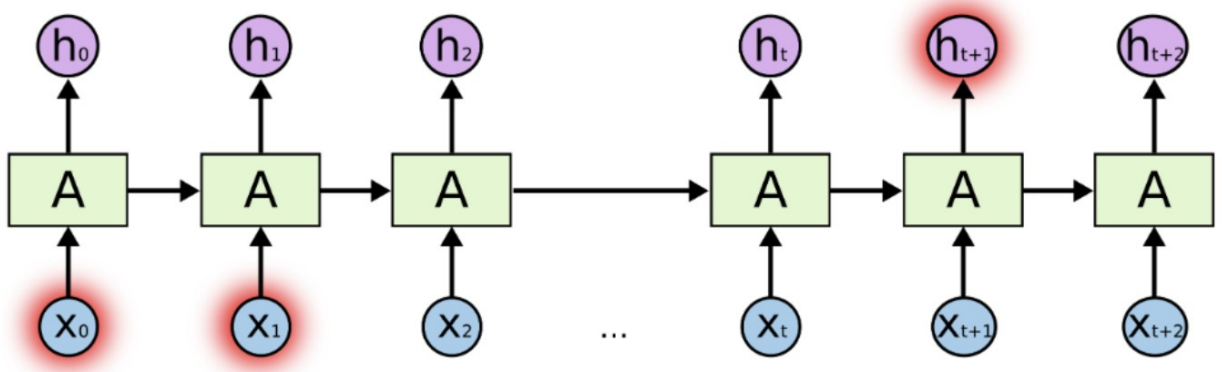
Summary

- RNN for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \cdots, \mathbf{x}_t$.



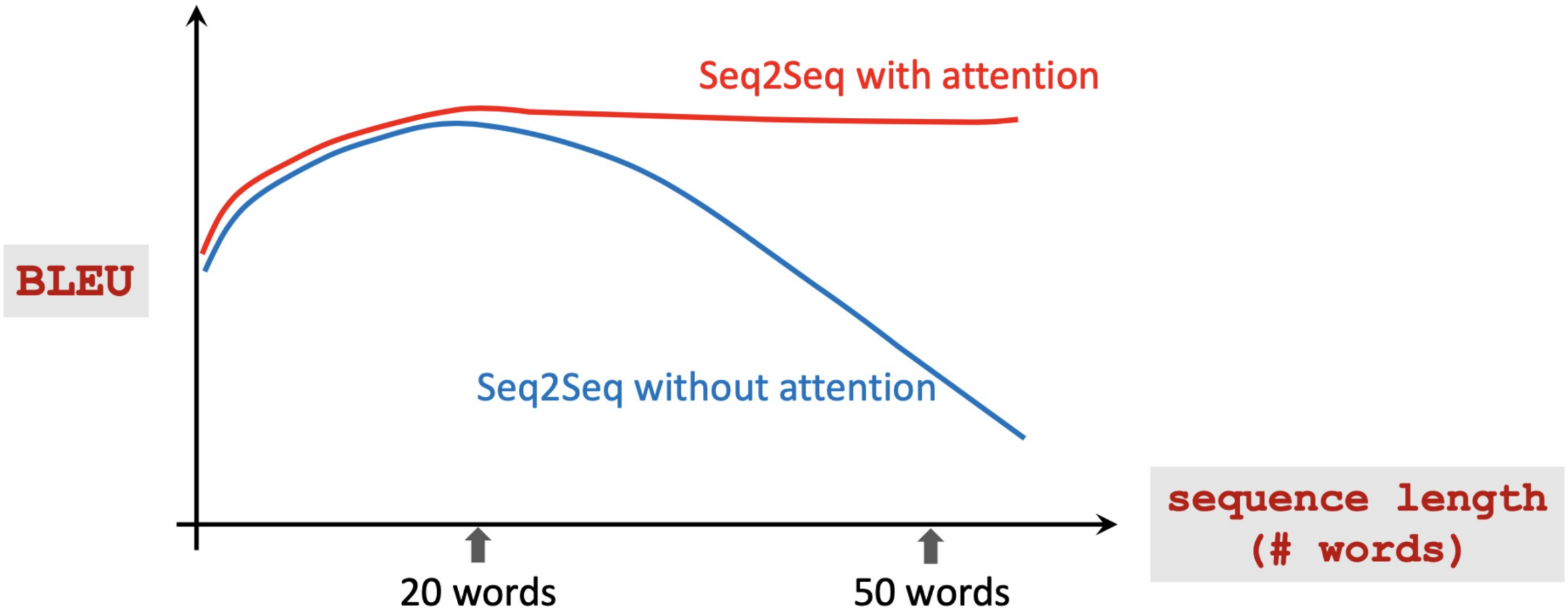
Summary

- RNN for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \cdots, \mathbf{x}_t$.
- RNNs can forget early inputs.
  - It forgets what it has seen early on.
  - If t is large, $\mathbf{h}_t$ is almost irrelevant to $\mathbf{x}_0$.



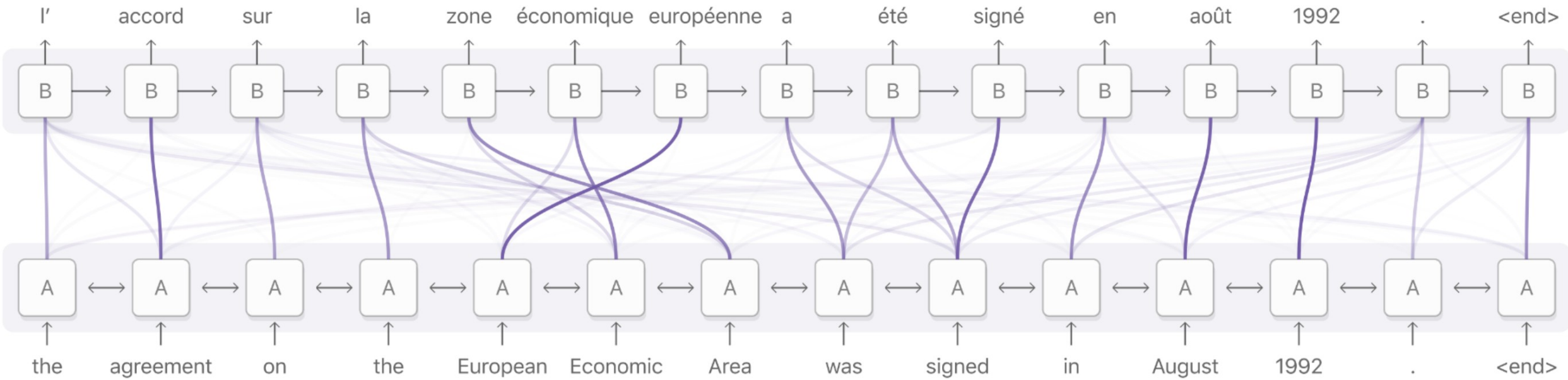
Seq2Seq Model

Shortcoming: The final state is incapable of remembering a **long** sequence.



Attention: Weights Visualization

Decoder RNN (target language: French)

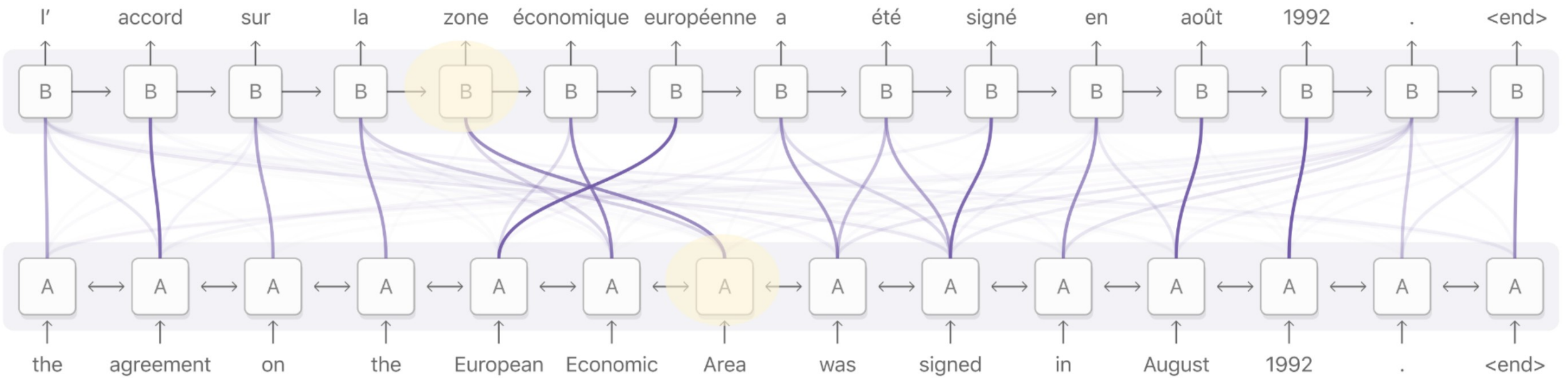


Encoder RNN (source language: English)

Figure is from https://distill.pub/2016/augmented-rnns/

Attention: Weights Visualization

Decoder RNN (target language: French)



Encoder RNN (source language: English)

Figure is from https://distill.pub/2016/augmented-rnns/

Seq2Seq Model with Attention

- Attention tremendously improves Seq2Seq model.
- With attention, Seq2Seq model does not forget source input.
- With attention, the decoder knows where to focus.
- Downside: much more computation.

Original paper:

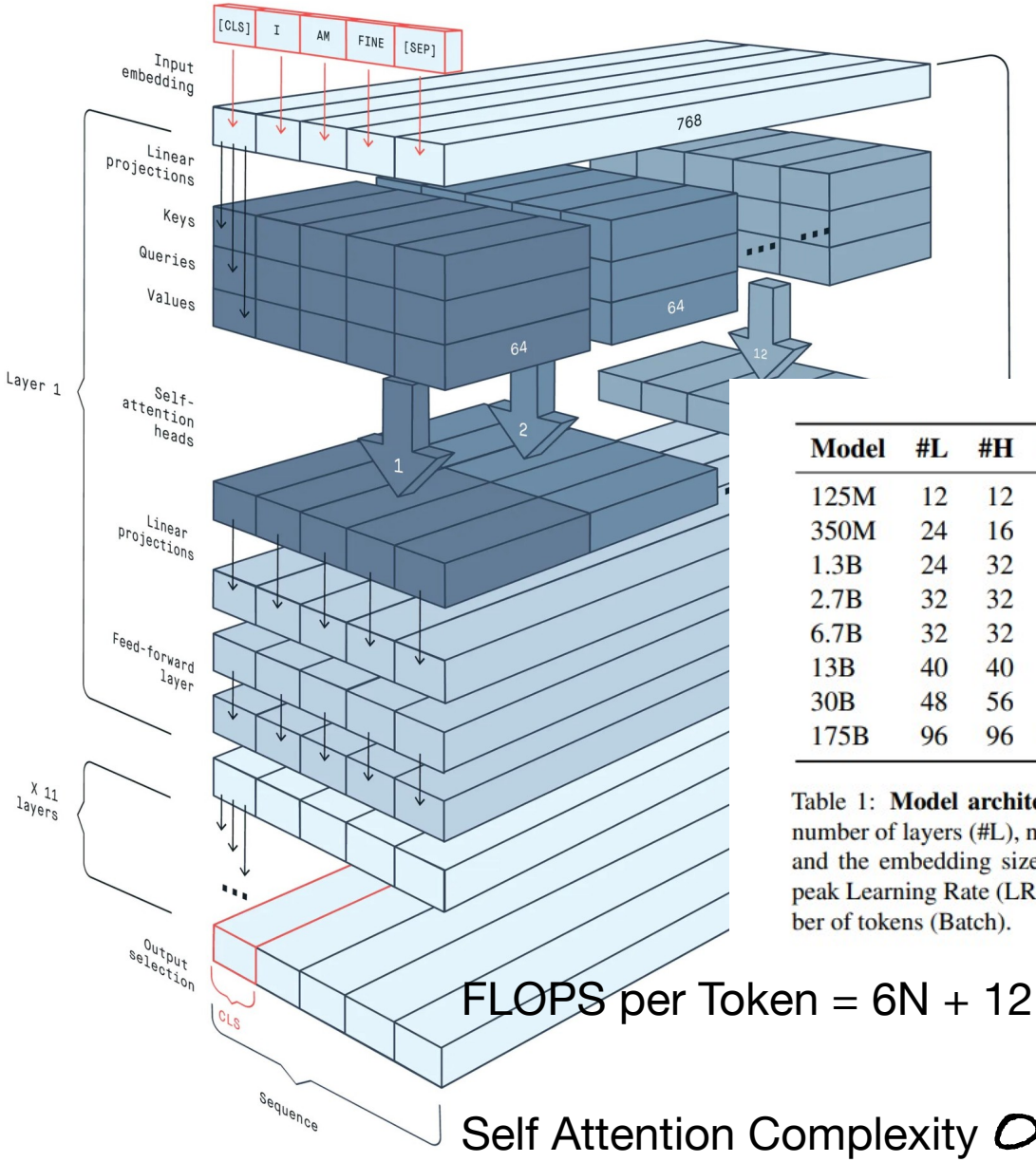
- Bahdanau, Cho, & Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

Summary

- Standard Seq2Seq model: the decoder looks at only its current state.
- Attention: decoder additionally looks at all the states of the encoder.
- Attention: decoder knows where to focus.

- Downside: higher time complexity.
  - m: source sequence length
  - t: target sequence length
  - Standard Seq2Seq: $O(m + t)$ time complexity
  - Seq2Seq + attention: $O(mt)$ time complexity

The Transformer with Self-Attention

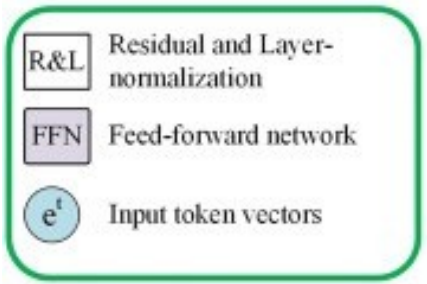


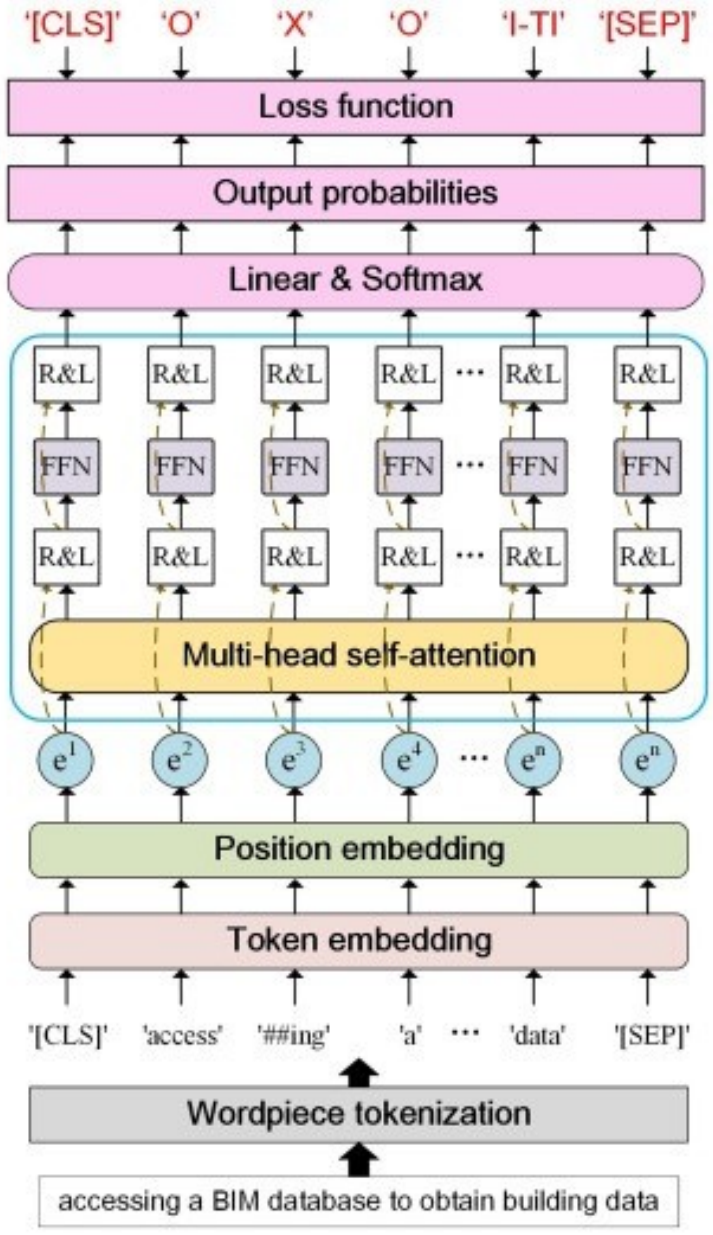
Model	#L	#H	d_{model}	LR	Batch
125M	12	12	768	$6.0e{-4}$	0.5M
350M	24	16	1024	$3.0e{-4}$	0.5M
1.3B	24	32	2048	$2.0e{-4}$	1M
2.7B	32	32	2560	$1.6e{-4}$	1M
6.7B	32	32	4096	$1.2e{-4}$	2M
13B	40	40	5120	$1.0e{-4}$	4M
30B	48	56	7168	$1.0e{-4}$	4M
175B	96	96	12288	$1.2e{-4}$	2M

Table 1: **Model architecture details.** We report the number of layers (#L), number of attention heads (#H), and the embedding size (d_{model}). We also report the peak Learning Rate (LR) and global batch size in number of tokens (Batch).

FLOPS per Token = 6N + 12LHQT

Self Attention Complexity $O(n^2d + nd^2)$





Layers, Heads, Dimension, Width

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Transformer

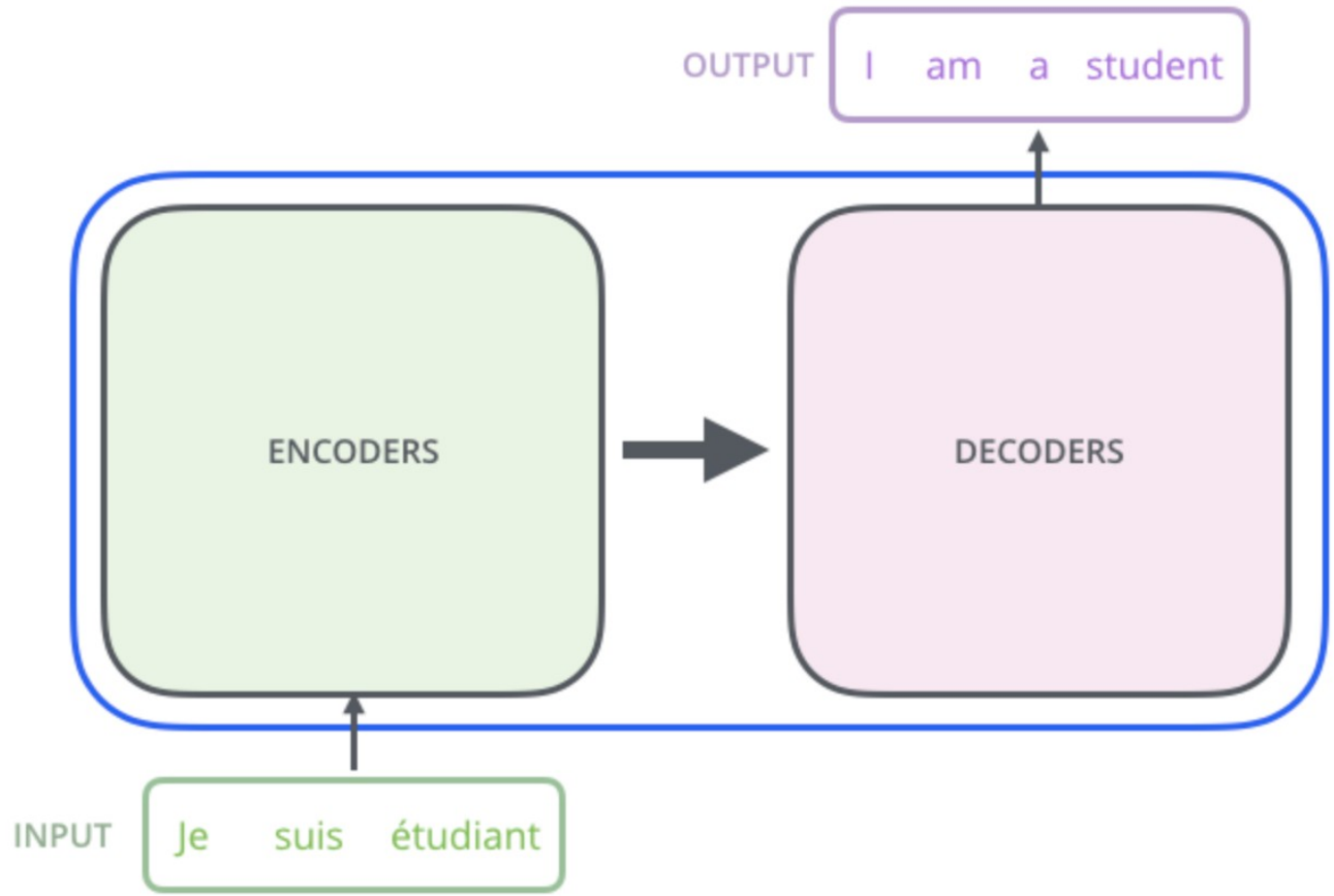
"Attention is All you Need" (Vaswani, et al., 2017), without a doubt, is one of the most impactful and interesting paper in 2017. It presented a lot of improvements to the soft attention and make it possible to do seq2seq modeling *without* recurrent network units. The proposed "**transformer**" model is entirely built on the self-attention mechanisms without using sequence-aligned recurrent architecture.

A High-Level Look

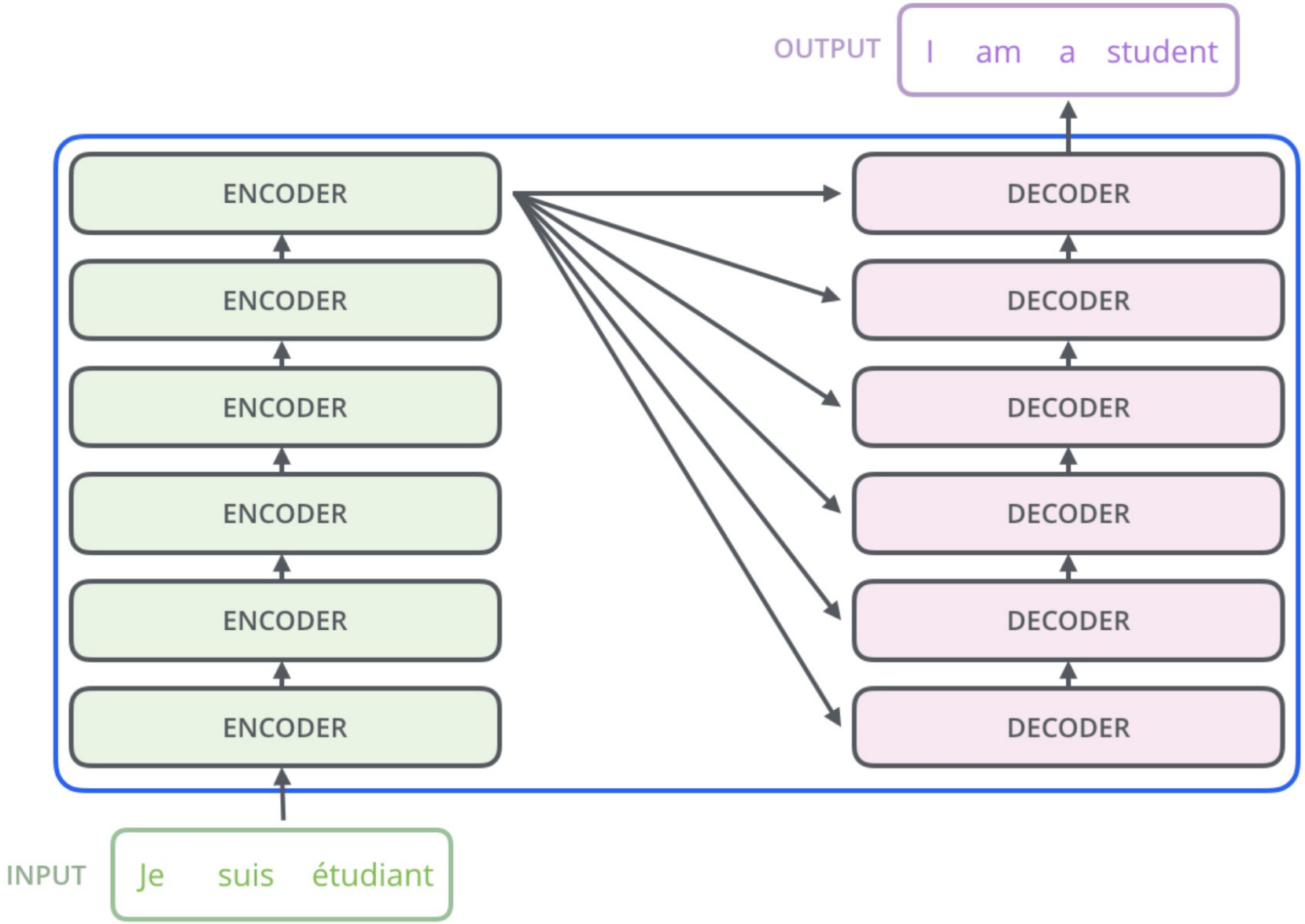
Let's begin by looking at the model as a single black box. In a machine translation application, it would take a sentence in one language, and output its translation in another.



Popping open that Optimus Prime goodness, we see an encoding component, a decoding component, and connections between them.



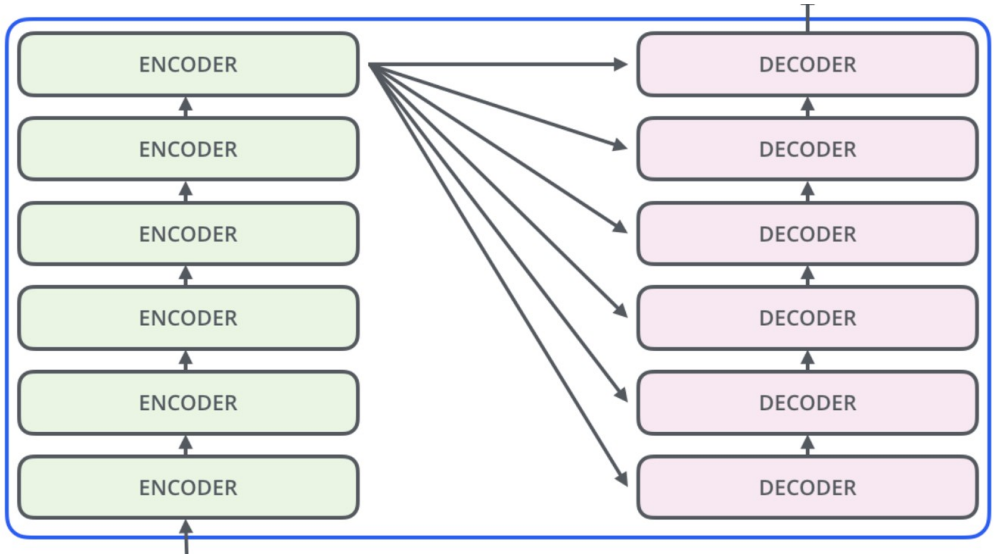
The encoding component is a stack of encoders (the paper stacks six of them on top of each other – there's nothing magical about the number six, one can definitely experiment with other arrangements). The decoding component is a stack of decoders of the same number.



Encoder-Decoder architecture

- Sequence to sequence models
- Encoder can see whole input (in the window)
- Decoder can see only to the left
- New sentences, summarization, translation, generative question answering

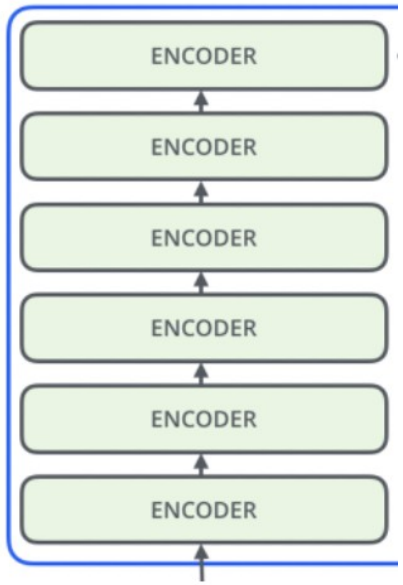
  - BART
  - mBART
  - Marian
  - T5



Encoder only, Decoder only

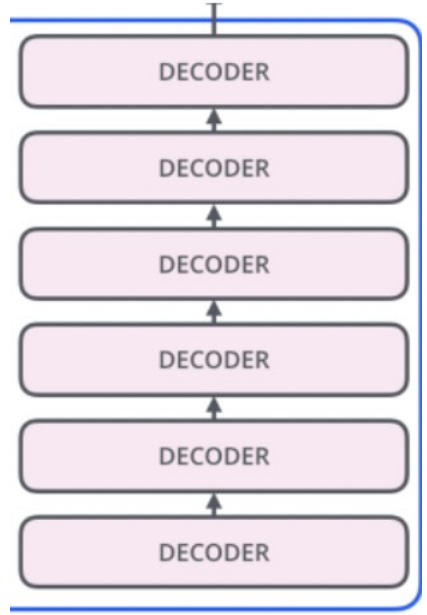
- Encoder only models
- Bidirectional
- Auto-encoding
- Classification, NER, Extractive QA

- ALBERT
- BERT
- DistilBERT
- ELECTRA
- RoBERTa

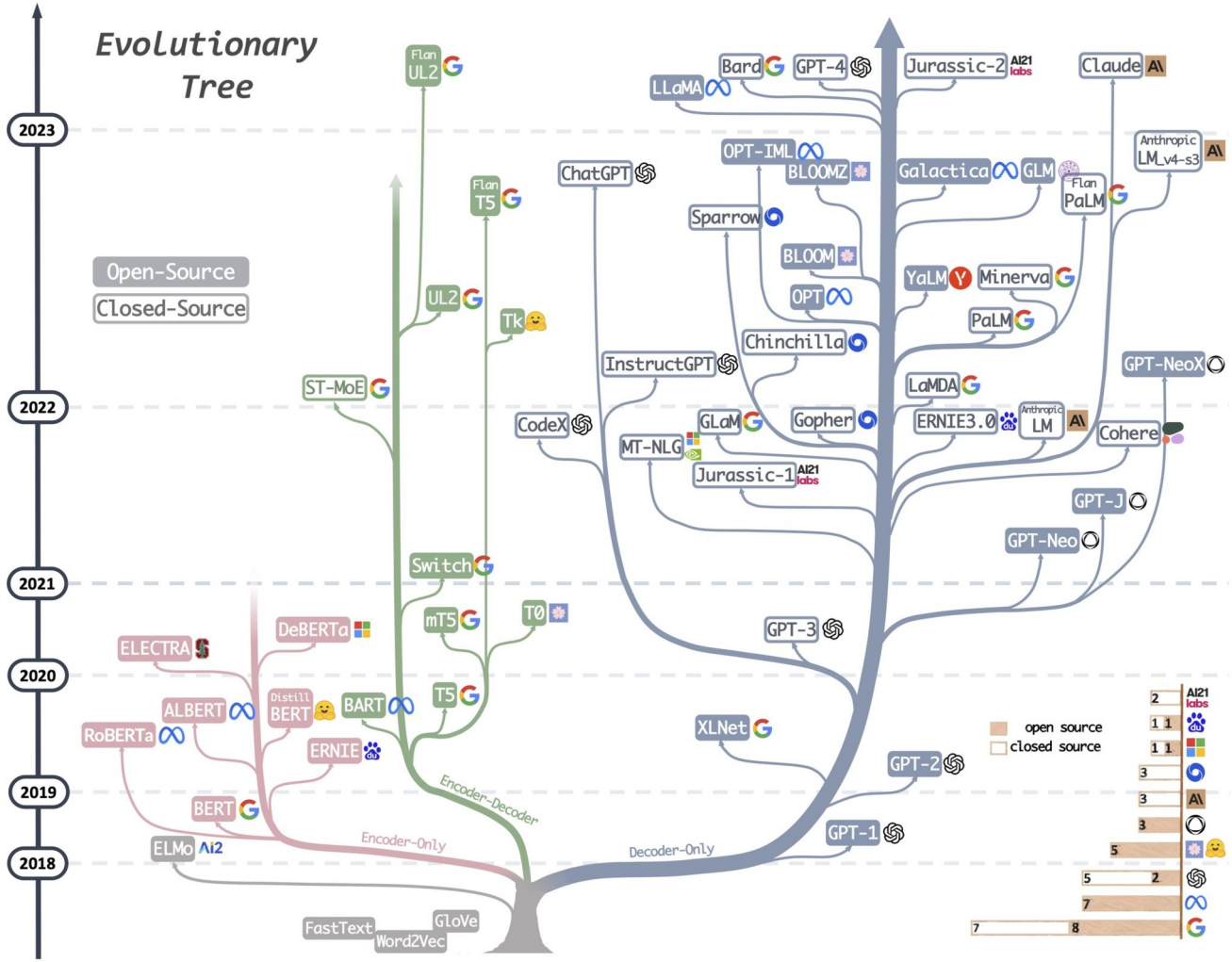


- Decoder only models
- Unidirectional
- Auto-regressive
- Text generation

- CTRL
- GPT
- GPT-2
- Transformer XL



Rapid Development of Large Language Models



Explosion of development of LLM based AI systems since 2019

Foundation Models are replacing narrow AI systems at a rapid pace

Foundation Models are the closest things that have yet been created that hint at the possibility of Artificial General Intelligence

Model	Examples	Tasks
Encoder	ALBERT, BERT, DistilBERT, ELECTRA, RoBERTa	Sentence classification, named entity recognition, extractive question answering
Decoder	CTRL, GPT, GPT-2, Transformer XL	Text generation
Encoder-decoder	BART, T5, Marian, mBART	Summarization, translation, generative question answering

The rise of decoder-only Transformer models

Apart from the various interesting features of this model, one feature that catches the attention is its decoder-only architecture. In fact, not just PaLM, some of the most popular and widely used language models are decoder-only.

GPT-3 has demonstrated that large autoregressive language models can be used for few-shot predictions, and this class of models is generally trained with decoder-only architecture and a standard left-to-right language modelling objective on a large text corpus where the objective is to predict the next token, taking into account the previous tokens. Not just GPT-3, the previous versions, GPT and GPT-2, too, utilised a decoder only architecture.

Subsequently, researchers from Google Brain published a paper titled – Generating Wikipedia by Summarising Long Sequences. The study proposed another arrangement of transformer blocks for language modelling. This model omitted the encoder block. For this research, the team introduced a new decoder-only sequence transduction model for the abstractive stage. They demonstrated that the model is capable of handling very long input-output examples. This model outperformed traditional encoder-decoder architectures on long sequences, allowing the team to condition on many reference documents and generate coherent and informative Wikipedia articles.